

Programación 1 — Unidad 5: Listas

Actividad 3: Acceso y modificación de listas

Índice

- 5.3. Modificar listas
- 5.4. Recorrer una lista
- 5.5. Rebanado (slicing)
- 5.6. Más métodos y funciones útiles
- 5.7. Recorrer con enumerate
- 5.8. Cuidado al copiar listas
- 5.9. Listas de listas (matrices)
- 5.10. Buscar vs. acceder: el costo de recorrer una lista

5.3. Modificar listas

Las listas son **mutables**: se pueden cambiar después de crearlas. Podemos reemplazar un elemento, agregar nuevos o eliminar existentes. Para muchas de estas operaciones usamos **métodos**: funciones que pertenecen a la lista y se llaman con un punto.

Operación	Qué hace	Ejemplo
<code>lista[i] = x</code>	Reemplaza el elemento en la posición <code>i</code>	<code>compras[0] = "agua"</code>
<code>.append(x)</code>	Agrega <code>x</code> al final	<code>compras.append("café")</code>
<code>.insert(i, x)</code>	Inserta <code>x</code> en la posición <code>i</code>	<code>compras.insert(1, "yerba")</code>
<code>.remove(x)</code>	Elimina la primera aparición de <code>x</code>	<code>compras.remove("sal")</code>
<code>.pop(i)</code>	Saca y devuelve el elemento <code>i</code>	<code>ultimo = compras.pop()</code>
<code>len(lista)</code>	Devuelve la cantidad de elementos	<code>len(compras)</code>

```
compras = ["pan", "leche"]
compras.append("huevos")
compras.insert(0, "agua")
compras.remove("leche")
print(compras)          # ['agua', 'pan', 'huevos']
print(len(compras))     # 3
```

Qué hace el código:

- `compras.append("huevos")` agrega ese texto al final: la lista pasa a tener tres elementos.

- `compras.insert(0, "agua")` mete **agua** en la posición 0 (el principio), corriendo el resto.
- `compras.remove("leche")` busca y elimina la primera aparición de **leche**.
- `len(compras)` cuenta cuántos elementos quedaron: en este caso, 3.

5.4. Recorrer una lista

La combinación de listas y bucles es una de las herramientas más potentes de toda la programación. Con **for** recorremos cada elemento de la lista, uno por uno, para hacer algo con cada uno: mostrarlos, sumarlos, contarlos.

```
notas = [7, 9, 4, 8, 10]

total = 0
for nota in notas:
    total += nota
promedio = total / len(notas)
print(f"Promedio: {promedio}")
```

Qué hace el código:

- `for nota in notas:` recorre la lista: en cada vuelta, **nota** toma el valor de un elemento (7, después 9, etc.).
- `total += nota` va acumulando: suma cada nota al total, que arrancó en 0.
- Al terminar el bucle, **total** tiene la suma de todas las notas.
- `total / len(notas)` divide la suma por la cantidad de notas para obtener el promedio.

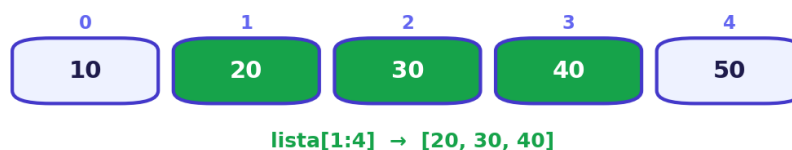
Analogía

Una lista es como un tren de vagones numerados desde 0. Cada vagón guarda una carga; podés ir directo al vagón que quieras por su número, agregar vagones al final o sacar alguno del medio. El bucle **for** es el inspector que recorre vagón por vagón, de principio a fin.

5.5. Rebanado (slicing)

El **rebanado** permite obtener un **trozo** de una lista (o de un texto) indicando un rango con la forma `lista[inicio:fin]`. Toma desde la posición *inicio* hasta **una antes** de *fin*. Si se omite un extremo, va hasta el principio o el final.

Rebanado (slicing): `lista[inicio:fin]` toma desde inicio hasta fin-1



El rebanado lista[1:4] devuelve los elementos de las posiciones 1, 2 y 3 (no incluye la 4).

```
numeros = [10, 20, 30, 40, 50]
print(numeros[1:4])    # [20, 30, 40]
print(numeros[:3])     # [10, 20, 30]
print(numeros[2:])     # [30, 40, 50]
print(numeros[::-1])   # [50, 40, 30, 20, 10]
```

Qué hace el código:

- `numeros[1:4]` toma desde la posición 1 hasta la 3 (una antes del 4): da `[20, 30, 40]`.
- `numeros[:3]` omite el inicio, así que empieza desde el principio: `[10, 20, 30]`.
- `numeros[2:]` omite el final, así que llega hasta el último: `[30, 40, 50]`.
- `numeros[::-1]` es un truco muy usado para invertir: devuelve la lista al revés.



Diagrama 5.5 generado con Mermaid.

5.6. Más métodos y funciones útiles

Python trae varias funciones y métodos que resuelven tareas comunes sobre listas sin que tengamos que escribir el bucle a mano:

Función / método	Qué hace	Ejemplo
<code>sum(lista)</code>	Suma todos los elementos	<code>sum([1,2,3]) → 6</code>
<code>max(lista) / min(lista)</code>	Mayor / menor elemento	<code>max([4,9,2]) → 9</code>
<code>.sort()</code>	Ordena la lista (la modifica)	<code>numeros.sort()</code>
<code>.reverse()</code>	Invierte el orden	<code>numeros.reverse()</code>
<code>.count(x)</code>	Cuenta apariciones de x	<code>[1,1,2].count(1) → 2</code>

```
notas = [7, 4, 10, 8, 6]
print(sum(notas))    # 35
print(max(notas))    # 10
notas.sort()
print(notas)         # [4, 6, 7, 8, 10]
```

Qué hace el código:

- `sum(notas)` suma todos los elementos de un saque: 35.
- `max(notas)` devuelve el mayor: 10. (Existe `min()` para el menor).
- `notas.sort()` reordena la lista de menor a mayor. Ojo: modifica la lista original.
- Por eso, al imprimir `notas` después, ya aparece ordenada.

5.7. Recorrer con enumerate

A veces, al recorrer una lista, necesitamos el elemento **y** su posición a la vez. La función `enumerate()` nos da los dos juntos, de forma más limpia que andar calculando el índice por separado.

```
compras = ["pan", "leche", "huevos"]
for indice, item in enumerate(compras):
    print(f"{indice}: {item}")
```

Qué hace el código:

- `enumerate(compras)` entrega, en cada vuelta, un par de valores: la posición y el elemento.
- `for indice, item in ...` los recibe en dos variables a la vez: `indice` y `item`.
- Así, en la primera vuelta `indice` vale 0 e `item` vale `pan`, y se muestra `0: pan`.
- Es más claro y menos propenso a errores que usar `range(len(compras))` y acceder con corchetes.

5.8. Cuidado al copiar listas

Hay un punto que confunde mucho al principio: asignar una lista a otra variable **no** crea una copia. Las dos variables terminan apuntando a la **misma** lista, así que modificar una afecta a la otra. Para copiar de verdad, se usa el método `.copy()`.

```
original = [1, 2, 3]
espejo = original
espejo.append(4)
print(original)          # [1, 2, 3, 4] ¡cambió!

copia = original.copy()
copia.append(5)
print(original)          # [1, 2, 3, 4] (no cambió)
```

Qué hace el código:

- `espejo = original` NO copia la lista: hace que ambos nombres apunten a la misma caja.
- Por eso, al hacer `espejo.append(4)`, también «cambia» `original`: son la misma lista.
- `original.copy()` sí crea una lista nueva e independiente, que guardamos en `copia`.
- Al modificar `copia`, la `original` queda intacta, porque ahora son listas distintas.

Importante

Este es uno de los errores más confusos al empezar. Recordá la regla: el signo `=` con listas **comparte** la misma caja, no la duplica. Para una copia real e independiente, usá `.copy()`.

5.9. Listas de listas (matrices)

Un elemento de una lista puede ser... otra lista. Así se representan tablas o **matrices**, como una grilla de filas y columnas. Para llegar a un elemento usamos dos índices: uno para la fila y otro para la columna.

```
tablero = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9],  
]  
print(tablero[0][2]) # 3  
print(tablero[2][0]) # 7
```

Qué hace el código:

- `tablero` es una lista que contiene tres listas adentro: cada una es una fila.
- `tablero[0]` accede a la primera fila completa: `[1, 2, 3]`.
- `tablero[0][2]` agrega un segundo índice: de la primera fila, toma la posición 2, que es 3.
- `tablero[2][0]` toma la tercera fila (`[7, 8, 9]`) y su primer elemento: 7.

Encontrá el error

Muchos esperan que `original` imprima `[1, 2, 3]`, sin el 4. Pensá qué significa realmente la línea `copia = original` antes de seguir.

```
original = [1, 2, 3]  
copia = original  
copia.append(4)  
print(original)
```

Qué estaba mal

`copia = original` no crea una lista nueva: copia el nombre por dirección de memoria, así que `copia` y `original` apuntan a la misma lista. Modificar una modifica la otra. Para copiar de verdad hay que usar `copia = original.copy()` o `copia = list(original)`.

Practicá la lectura

Sin ejecutar, anotá qué imprime cada línea, prestando atención a qué elementos incluye y cuáles no incluye el rebanado (slicing).

```
numeros = [10, 20, 30, 40, 50]  
print(numeros[1:4])  
print(numeros[:2])  
print(numeros[-2:])
```

Para reflexionar y autoevaluarte

— ¿Te pasó de esperar que una lista quedara intacta y en realidad haberla modificado por accidente?

— ¿Cuándo te conviene más usar índices numéricos y cuándo un for elemento in lista?

Antes de seguir, verificá que podés:

- Acceder y modificar elementos de una lista por índice, incluidos índices negativos.
- Recorrer una lista con for y con enumerate.
- Usar slicing para obtener partes de una lista.
- Explicar la diferencia entre copiar una lista y copiar su referencia.

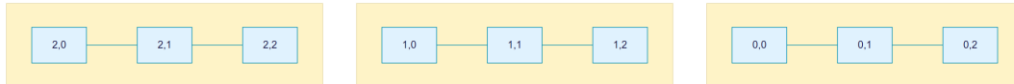


Diagrama 5.9 generado con Mermaid.

5.10. Buscar vs. acceder: el costo de recorrer una lista

En la Unidad 3 vimos que dos programas pueden resolver el mismo problema con distinto costo. Esa misma idea aparece al trabajar con listas.

```
numeros = [4, 8, 15, 16, 23, 42]
print(23 in numeros)
print(numeros[3])
```

Buscar un valor con `in` (o con un bucle propio) obliga a Python, en el peor caso, a revisar **todos** los elementos de la lista hasta encontrarlo o llegar al final. Cuanto más grande la lista, más puede tardar.

Acceder por índice (`numeros[3]`) en cambio va directo a esa posición: cuesta lo mismo sin importar si la lista tiene 6 elementos o 6 millones.

Analogía

Es la misma diferencia que hay entre buscar un libro en un estante desordenado, título por título, y tomar el libro que ya sabés que está en la tercera balda: en el segundo caso vas directo, sin revisar nada más.

No hace falta memorizar ninguna fórmula todavía: alcanza con desarrollar el reflejo de preguntarse, frente a una lista grande, si la operación que se necesita hacer va a tener que recorrerla entera o no.